informs
https://pubsonline.informs.org/journal/opre

OPERATIONS RESEARCH

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

Submitted to *Operations Research*

ISSN 0030-364X, EISSN 1526-5463

Robust Candidate MaxWeight Certificates for Heterogeneous Compute Scheduling: A Scheduleurm Case Study

(Authors' names are not included for peer review)

Abstract. Heterogeneous compute clusters run workloads whose service rates depend on co-location, host pressure, accelerator memory, and topology. We model this as a configuration-action stochastic processing network and study a robust candidate MaxWeight policy that keeps the full action space in the capacity statement while charging candidate approximation, conservative service estimation, penalties, and approximate optimization against explicit slack. If the residual slack margin is positive, the policy satisfies a finite-set Foster recurrence certificate under bounded conditional second moments; the fixed-family and statewise variants are formalized in Lean. The main contribution is a proof and certification framework for approximate configuration-action scheduling, not an exhaustive systems race. We instantiate the framework in Scheduleurm to show how a real scheduler can produce the finite service maps, lower-service certificates, oracle-gap records, and admission boundaries required by the theorem. On audited measured slices and 192 Poisson/bursty replay scenarios, the robust candidate improves legacy completion metrics while preserving explicit claim boundaries. Comparisons to external schedulers are reported as policy-semantics diagnostics on the same measured service cache and adapter audits; they are not used as direct full-stack superiority claims.

Key words: stochastic processing networks; MaxWeight scheduling; heterogeneous clusters; graphics-processing-unit co-location; robust optimization; formal verification; queueing networks

Subject classifications: Queues: scheduling; stochastic models: processing networks; computing: resource allocation

Area of review: Stochastic Models; Computing

1. Introduction

Schedulers for modern research clusters operate in a regime that is poorly described by a fixed resource vector. A reinforcement learning (RL) job can retain near-solo progress when several copies share a graphics processing unit (GPU), whereas a dense GPU kernel can lose finite-batch

delay performance when the same GPU is aggressively multiplexed. Central processing unit (CPU)-heavy evaluation jobs stress host cores and memory rather than accelerator compute, and short control-plane jobs expose queueing overhead and fragmentation. These cases are not exceptions to a simple model; they are the model. The scheduler chooses global configurations whose service rates depend on the set of jobs, their placement, co-location profile, and local hardware state.

The operations-research difficulty is that the natural action is a global configuration. Enumerating all placements and co-location patterns is combinatorial, but reducing the problem to a small list of hand-picked "sweet spots" can invalidate stability claims. A policy may look efficient on a finite batch while silently using an infeasible profile, or it may be throughput-optimal in a model that is disconnected from the measured service map. Our starting point is therefore to keep the full action family in the mathematical statement and to treat the implemented candidate family as an approximation whose loss must be paid from explicit slack. Figure 1 gives the one-page certificate structure: candidate construction, lower-service scoring, slack accounting, and bounded claim boundaries are checked as separate objects.

The main theoretical contribution is a robust candidate MaxWeight theorem with complete slack accounting. The theorem states exactly how candidate approximation, conservative service estimation, scheduling penalties, and approximate optimization consume capacity slack. When the remaining margin is positive and the stochastic primitives have bounded conditional second moments, the resulting policy satisfies a finite-set Foster recurrence certificate. The result is stated for both fixed feasible families and state-dependent feasible families, the latter being the case relevant to live schedulers whose feasible actions depend on available jobs, alive nodes, and measured capacity boundaries.

The empirical contribution is a theorem-condition audit, not a claim that a single implementation has solved cluster scheduling in full generality. We instantiate the theorem in Scheduleurm, an existing scheduler used for local research workloads, and report bounded pieces of evidence. First, measured service curves close declared finite slices for four workload buckets: local light-control tasks, GPU-heavy JAX numerical-computing matrix-multiplication tasks, local CPU-heavy tasks, and Robust-Ensemble Soft Actor-Critic (RE-SAC) / Bayesian Amnesic Piecewise-Robust (BAPR)-like hybrid RL tasks. Second, explicit replay compares robust candidate actions with legacy rules and diagnostic policy families on a common measured service cache, so all policies are evaluated against the same lower-service object. Third, holdout diagnostics, finite lower-service capacity certificates, ablation, production coverage, and live-state dry-run trace audits identify

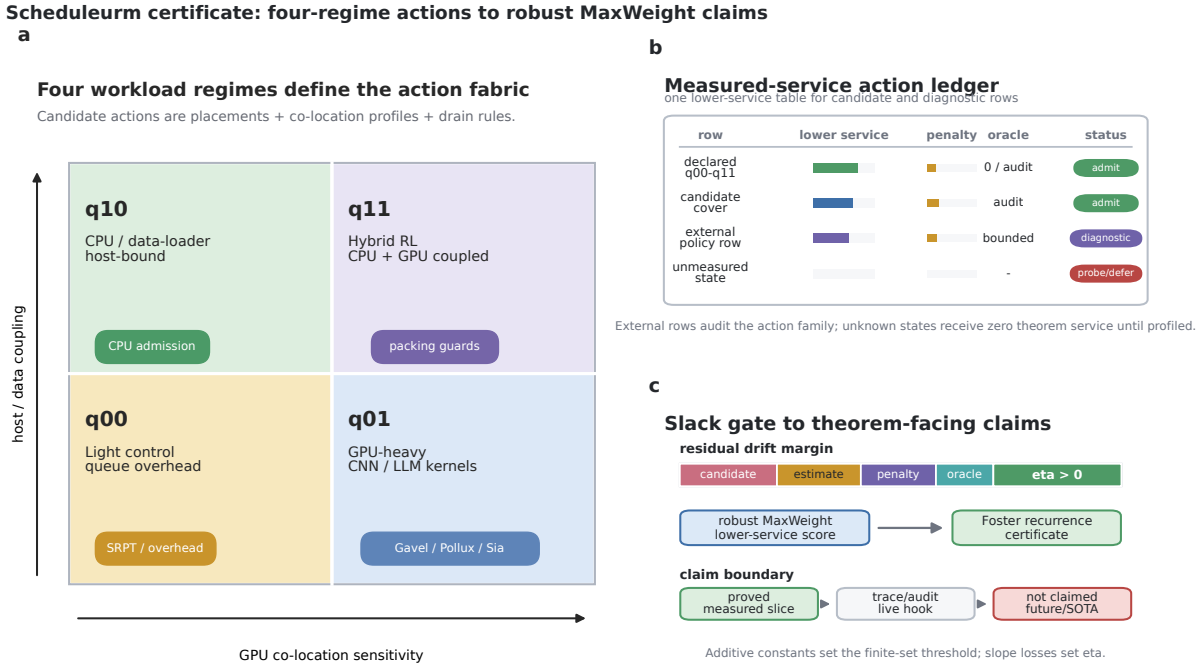

Figure 1 Overview of the robust candidate MaxWeight certificate. A combinatorial full action space is represented by measured finite candidate actions across four workload quadrants. The scheduler scores candidates using the queue-weighted lower-service objective minus bounded penalties, and the stability certificate requires the residual slack to exceed candidate-cover loss, service-estimation error, penalty growth, and approximate-oracle error. External policy families are diagnostic finite actions on the same service cache, not universal full-stack baselines. The measured-service action ledger is a visual summary of measured lower-service rows, bounded penalties, oracle audits, and admission status.

which assumptions are certified and which remain finite-slice claims. These results connect the proof obligations to scheduler artifacts; they do not replace a universal fabric-cover theorem or an exhaustive full-stack scheduler competition.

The paper is organized as follows. We first state the claim scope and reviewer-facing evidence contract. We then introduce the configuration-action processing network, give the candidate approximation and robust MaxWeight stability theorem, describe the Scheduleurm instantiation, report the computational evidence, position the work relative to scheduler and queueing literatures, and close with the remaining calibration limits.

2. Scope and Contributions

Table 1 states the reviewer-facing claim boundary used throughout the paper. The table is not a disclaimer appended after the fact; it is part of the methodology. Each computational claim is tied to the population and artifact that make it auditable, and each row also states the adjacent claim that is intentionally not made.

Table 1 Short claim matrix for scope, evidence, and non-claims.

Claim	Ready evidence	Not claimed
Robust MaxWeight stability theorem	Human proof and Lean artifact for exact and approximate robust candidate MaxWeight under stated slack, lower-service, penalty, oracle-error, and moment assumptions.	Automatic stability of an arbitrary production scheduler row.
Declared measured service domain	Measured q00/q01/q10/q11 slices, mixed portfolio replay, finite-slice slack certificate, declared finite-domain positive-cover gate, and conservative all-state safety gate.	Positive lower service for arbitrary future workloads or hardware states outside admission.
Live theorem hook and production bridge	Optional lower-service hook, strict theorem admission telemetry, bounded launched ScheduleurmBench trace, active-production shadow trace, strict scheduler-history organic completion certificate, and safe launch/completion gate.	A deployed global batch MaxWeight scheduler, raw-history closure, or automatic theorem admission for future unknown jobs.
External scheduler diagnostics	Policy-semantics replay on the same measured service cache, registered adapter rows, and scoped external-runtime audits used only to stress the finite action/certificate interface.	Exhaustive state-of-the-art (SOTA) superiority, arbitrary future workloads, or every external scheduler's original multi-node deployment.
Learning and regime extensions	Active-bucket, switching, and detector certificates, plus opt-in audit fields.	Default live adaptive learning with A/B performance evidence.

Appendix Table 15 gives the full gate ladder with status fields, blockers, thresholds, and artifact paths.

3. Model

3.1. Notation

Table 2 fixes the symbol ledger used throughout the paper. Each mathematical symbol is reserved for the meaning shown in the table; in particular, Q is the queue vector, q is an arbitrary nonnegative support weight, K_t is switching cost, and G_t is the risk or interference penalty.

3.2. Configuration actions

Let $\mathcal{I}$ be a finite set of job classes. Queue $Q_i(t) \in \mathbb{Z}_+$ counts the amount of unfinished work of class i at the beginning of slot t. A global configuration action $a \in \mathcal{A}^{\text{full}}$ specifies which jobs are served, which resource configuration each job receives, and where each job is placed. In a GPU cluster this includes, for example, the number of jobs sharing a GPU, CPU worker counts, memory pressure buckets, and host or device placement. The model intentionally treats these as one action rather than as independent per-job choices.

Given a regime z and exogenous randomness ω, action a generates a nonnegative service vector $S(a, z, \omega) \in \mathbb{R}_+^{|\mathcal{I}|}$. We write $\mu^z(a) = \mathbb{E}[S(a, z, \omega)]$. For the main theorem we first suppress z and use $\mu(a)$; uniform-in-regime and average-regime extensions require additional switching assumptions and are not used as main empirical claims.

Table 2 Notation ledger.

Symbol	Meaning
$\mathcal{I}, i$	finite set of job classes and a class index
$t, Q_i(t), Q(t)$	slot, class-i backlog, and queue vector
λ, v	arrival mean vector and feasible service vector
a, a_t	configuration action and chosen action
$\mathcal{A}^{\text{full}}, \mathcal{A}^{\text{cand}}$	full action family and fixed candidate family
$\mathcal{A}_t^{\text{cand}}, \mathcal{A}^{\text{meas}}$	statewise candidate family and measured action slice
$Z_t, \chi(t), \omega_t, A_i(t), S_i(t)$	hidden regime, scheduler-visible state, exogenous randomness, class-i arrival, and class-i service
$\mu^z(a), \mu(a), \mu_t^{\text{L}}(a)$	regime service mean, main-theorem service mean, and conservative lower-service estimate
$C(\mathcal{A}), \Lambda(\mathcal{A}), \text{Mix}(\mathcal{A}), x_a$	service convex hull, downward-closed load region, probability simplex, and stationary mixing mass on action a
$H_{\mathcal{A}}(q), q$	support function and nonnegative support vector
$\Phi, d_\Phi, w_r, L, \rho, \pi$	finite action features, fabric metric, feature weight, service Lipschitz envelope, cover radius, and candidate projection
$K_t(a), G_t(a), P_t(a), P_0, \beta$	switching cost, risk/interference penalty, total penalty, additive penalty bound, and queue-scaled penalty coefficient
θ_{prof}	dimensionless profile-penalty fraction used by the queue-adaptive replay candidate
$\delta, \epsilon_{\text{cand}}, \epsilon_{\text{est}}, \alpha_0, \alpha_1, \eta$	full-region slack, candidate loss, estimation loss, additive oracle error, queue-scaled oracle error, and residual drift margin
$B, N, V, \Delta V$	second-moment bound, finite-set threshold, quadratic Lyapunov function, and one-step Lyapunov drift
$b, k, \widehat{\mu}_i(k), \widehat{\mu}_i^{\max}$	replay backlog count, measured co-location profile, measured aggregate service at profile k, and best measured aggregate service in the same envelope

The notation avoids reusing one symbol for multiple meanings. Acronyms are expanded at first textual use and then used consistently.

The queue dynamics are

$$Q_i(t+1) = [Q_i(t) - S_i(t)]^+ + A_i(t), \qquad i \in \mathcal{I}, \tag{1}$$

where $A_i(t)$ is the class-i arrival in slot t. In the finite-support specialization used by the formal proof, the conditional distribution of $(A(t), S(t))$ depends on $Q(t)$ through a finite sample space. In the more general theorem it is enough to assume bounded conditional second moments.

3.3. Capacity and support functions

For a finite action family $\mathcal{A}$, define the service region

$$C(\mathcal{A}) = \left\{ \sum_{a \in \mathcal{A}} x_a \mu(a) : x \in \text{Mix}(\mathcal{A}) \right\}, \tag{2}$$

and its downward closure

$$\Lambda(\mathcal{A}) = \left\{ \lambda \in \mathbb{R}_+^{|\mathcal{I}|} : \exists v \in C(\mathcal{A}) \text{ with } \lambda \le v \right\}. \tag{3}$$

The support function in the nonnegative orthant is

$$H_{\mathcal{A}}(q) = \max_{a \in \mathcal{A}} q^\top \mu(a), \qquad q \in \mathbb{R}_+^{|I|}. \tag{4}$$

Here $\mathrm{Mix}(\mathcal{A})$ is the probability simplex over the action family $\mathcal{A}$, and x_a is the stationary mixing mass assigned to action a. The vector q is a nonnegative support weight; it is not the queue state unless explicitly specialized to $Q(t)$. If λ has coordinate slack $\delta > 0$ in the full region, then there is a stationary mixture $x \in \mathrm{Mix}(\mathcal{A}^{\mathrm{full}})$ such that $\lambda_i + \delta \le \sum_a x_a \mu_i(a)$ for every i. Consequently,

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\mathrm{full}}}(q), \qquad q \ge 0. \tag{5}$$

Inequality (5) is the bridge from capacity to MaxWeight drift; it is not an operational stability "if and only if" by itself.

3.4. Candidate families and fabric metrics

The implemented scheduler optimizes over $\mathcal{A}^{\mathrm{cand}} \subseteq \mathcal{A}^{\mathrm{full}}$. To avoid turning this computational restriction into an unaccounted model change, define a finite feature map $\Phi : \mathcal{A}^{\mathrm{full}} \to \mathbb{R}^d$ and weighted ℓ_1 fabric metric

$$d_\Phi(a, a') = \sum_{r=1}^{d} w_r |\Phi_r(a) - \Phi_r(a')|. \tag{6}$$

The features may include co-location profile, node bucket, CPU bucket, memory pressure bucket, placement locality, and other scheduler-visible fabric attributes. We say that $\mathcal{A}^{\mathrm{cand}}$ is a ρ-cover of $\mathcal{A}^{\mathrm{full}}$ if for each full action a there is a candidate action $\pi(a) \in \mathcal{A}^{\mathrm{cand}}$ with $d_\Phi(a, \pi(a)) \le \rho$. If $\|\mu(a) - \mu(a')\|_\infty \le L d_\Phi(a, a')$, then

$$H_{\mathcal{A}^{\mathrm{full}}}(q) \le H_{\mathcal{A}^{\mathrm{cand}}}(q) + L\rho \|q\|_1, \qquad q \ge 0. \tag{7}$$

The support-loss bound (7) is used as a theorem assumption unless a perturbation-profiling table certifies Φ, L, ρ. For exact measured finite slices in which the candidate family enumerates the measured actions, we set $\rho = 0$.

4. Robust Candidate MaxWeight

4.1. Policy

Let $\mu_t^{\mathrm{L}}(a)$ be a conservative lower-service estimate for candidate action a at time t. Let $K_t(a) = K(a_{t-1}, a)$ model switching or rollback cost from the previous action to a, and let $G_t(a)$ model risk or interference penalties. The exact robust candidate MaxWeight rule is

$$a_t \in \arg\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \big\{ Q(t)^\top \mu_t^{\mathrm{L}}(a) - K_t(a) - G_t(a) \big\}. \tag{8}$$

For practical schedulers, the selected action may be only approximately optimal. We assume the queue-scaled oracle condition relative to (8),

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \left\{ Q^\top \mu_t^{\mathrm{L}}(a) - K_t(a) - G_t(a) \right\} \\ &\quad - \left\{ Q^\top \mu_t^{\mathrm{L}}(a_t) - K_t(a_t) - G_t(a_t) \right\} \\ &\le \alpha_0 + \alpha_1 \left\| Q \right\|_1 . \end{aligned} \tag{9}$$

The oracle loss in (9) is paid from the same slack budget as candidate loss and lower-service error. Exact maximization is the special case $\alpha_0 = \alpha_1 = 0$.

4.2. Main stability certificate

4.2.1. Assumptions We state the assumptions explicitly because each one corresponds to a measurable artifact in Scheduleurm. All constants below are nonnegative and are fixed over the horizon under consideration.

Table 3 is the proof spine used below. The numbered lemmas give the main-text derivation; Appendix "Conventional Proof Details" then expands the same chain in electronic-companion style so that the reader can check the theorem without opening the Lean files.

ASSUMPTION 1 **(Load slack)**. *The arrival mean vector λ has full-action support slack $\delta > 0$:*

$$q^\top \lambda + \delta \left\| q \right\|_1 \le H_{\mathcal{A}^{\mathrm{full}}}(q), \qquad q \in \mathbb{R}_+^{|\mathcal{I}|}. \tag{10}$$

This is the support-function form of belonging to the interior of the full-action capacity base. It is a sufficient stability hypothesis; the operational necessity direction requires a separate conservation law.

Table 3 Proof roadmap and slack accounting for Theorem 1.

Step	Human-readable statement	Slack or condition consumed
Capacity support	A full-action stationary mixture with coordinate slack δ implies $q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q)$ for all nonnegative support vectors q.	No implementation loss; converts capacity slack into a support-function margin.
Candidate cover	The candidate family approximates the full family, giving $H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1$; under a calibrated fabric metric $\epsilon_{\text{cand}} = L\rho$.	Candidate-set or fabric-cover loss $L\rho$.
Lower service	Conservative service rows satisfy $H_{\mathcal{A}_t^{\text{cand}}}(q) \le \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) + \epsilon_{\text{est}} \|q\|_1$.	Service-estimation or LCB loss ϵ_{est}.
Penalty and oracle	Bounded penalties and approximate optimization imply that the selected action loses at most $P_0 + \alpha_0 + (\beta + \alpha_1) \|Q\|_1$ relative to the lower-service support.	Bounded penalty (P_0, β) and oracle error (α_0, α_1).
Drift margin	The selected action provides $Q^\top \mu_t^{\text{L}}(a_t) \ge Q^\top \lambda + \eta \|Q\|_1 - (P_0 + \alpha_0)$ with $\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1)$.	Requires residual margin $\eta > 0$.
Foster certificate	The quadratic Lyapunov inequality and bounded second moments give $\mathbb{E}[\Delta V \mid Q] \le B + P_0 + \alpha_0 - \eta \|Q\|_1$, hence negative drift outside a finite set.	Moment bound B, finite sublevel set, and local-return condition for positive recurrence.

The table is the human-readable proof spine. The Lean artifact checks the same algebraic implications, while the service, cover, oracle, and moment assumptions are certified by experiment gates rather than by Lean.

ASSUMPTION 2 **(Candidate cover and lower-service error)**. *For every scheduler-visible state t and every $q \in \mathbb{R}_+^{|\mathcal{I}|}$,*

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 , \tag{11}$$

and

$$H_{\mathcal{A}_t^{\text{cand}}}(q) \le \max_{a \in \mathcal{A}_t^{\text{cand}}} q^\top \mu_t^{\text{L}}(a) + \epsilon_{\text{est}} \|q\|_1 . \tag{12}$$

When the fabric metric in Section 3.4 is certified, $\epsilon_{\text{cand}} = L\rho$. For an exact measured finite slice, $\epsilon_{\text{cand}} = 0$.

ASSUMPTION 3 **(Penalty growth and approximate oracle)**. *Let $P_t(a) = K_t(a) + G_t(a)$. For all $a \in \mathcal{A}_t^{\text{cand}}$,*

$$0 \le P_t(a) \le P_0 + \beta \|Q(t)\|_1 . \tag{13}$$

The selected action $a_t \in \mathcal{A}_t^{\text{cand}}$ satisfies the approximate-oracle condition

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q(t)^\top \mu_t^{\text{L}}(a) - P_t(a) \right\} \\ &\quad - \left\{ Q(t)^\top \mu_t^{\text{L}}(a_t) - P_t(a_t) \right\} \le \alpha_0 + \alpha_1 \|Q(t)\|_1 . \end{aligned} \tag{14}$$

Exact robust MaxWeight is the special case $\alpha_0 = \alpha_1 = 0$.

ASSUMPTION 4 **(Stochastic domination and bounded second moments)**. *Conditional on $Q(t) = Q$,*

$$\mathbb{E}[A_i(t) \mid Q(t) = Q] \le \lambda_i, \tag{15}$$

and

$$\mu^{\mathrm{L}}_{t,i}(a_t) \le \mathbb{E}[S_i(t) \mid Q(t) = Q] \tag{16}$$

for every class i, and

$$\mathbb{E}\left[\frac{1}{2}\sum_i \{A_i(t)^2 + S_i(t)^2\} \,\middle|\, Q(t) = Q\right] \le B. \tag{17}$$

ASSUMPTION 5 **(Countable-state recurrence condition)**. *For a positive-recurrence conclusion, the induced controlled process is a Markov chain on a countable queue state space, sublevel sets $\{Q : \|Q\|_1 \le N\}$ are finite, and the usual local-return condition holds on the finite set selected by the drift certificate. Without this assumption we claim only the finite-set negative-drift certificate.*

4.2.2. Deterministic slack accounting

LEMMA 1 **(Support slack after candidate and estimation losses)**. *Under Assumptions 1–2, for every t and $q \in \mathbb{R}_+^{|\mathcal{I}|}$,*

$$\begin{aligned}\max_{a\in\mathcal{A}_t^{\mathrm{cand}}} q^\top \mu_t^{\mathrm{L}}(a) \ge\ & q^\top \lambda \\ & + (\delta - \epsilon_{\mathrm{cand}} - \epsilon_{\mathrm{est}})\, \|q\|_1 .\end{aligned} \tag{18}$$

Proof. By Assumption 1,

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\mathrm{full}}}(q). \tag{19}$$

Assumption 2 gives

$$\begin{aligned}H_{\mathcal{A}^{\mathrm{full}}}(q) &\le H_{\mathcal{A}_t^{\mathrm{cand}}}(q) + \epsilon_{\mathrm{cand}} \|q\|_1 \\ &\le \max_{a\in\mathcal{A}_t^{\mathrm{cand}}} q^\top \mu_t^{\mathrm{L}}(a) \\ &\quad + (\epsilon_{\mathrm{cand}} + \epsilon_{\mathrm{est}})\, \|q\|_1 .\end{aligned} \tag{20}$$

Rearranging proves the claim.

LEMMA 2 **(Selected lower-service margin)**. *Under Assumptions 1–3, for $Q = Q(t)$,*

$$\begin{aligned}Q^\top \mu_t^{\mathrm{L}}(a_t) \ge\ & Q^\top \lambda \\ & + \left(\delta - \epsilon_{\mathrm{cand}} - \epsilon_{\mathrm{est}} - \beta - \alpha_1\right) \|Q\|_1 \\ & - (P_0 + \alpha_0).\end{aligned} \tag{21}$$

Proof. Let $M(Q) = \max_{a \in \mathcal{A}_t^{\text{cand}}} Q^\top \mu_t^{\text{L}}(a)$. Since $P_t(a) \le P_0 + \beta \|Q\|_1$ for every candidate,

$$\begin{aligned} &\max_{a \in \mathcal{A}_t^{\text{cand}}} \{Q^\top \mu_t^{\text{L}}(a) - P_t(a)\} \\ &\quad \ge M(Q) - P_0 - \beta \|Q\|_1 . \end{aligned} \tag{22}$$

The approximate-oracle condition therefore implies

$$\begin{aligned} Q^\top \mu_t^{\text{L}}(a_t) - P_t(a_t) \ge M(Q) - P_0 - \beta \|Q\|_1 \\ - \alpha_0 - \alpha_1 \|Q\|_1 . \end{aligned} \tag{23}$$

Because $P_t(a_t) \ge 0$, the left-hand side is at most $Q^\top \mu_t^{\text{L}}(a_t)$. Applying Lemma 1 with $q = Q$ gives the stated inequality.

LEMMA 3 **(Quadratic one-step bound)**. *For $V(Q) = \frac{1}{2}\sum_i Q_i^2$ and nonnegative vectors Q, A, S,*

$$\begin{aligned} &V([Q - S]^+ + A) - V(Q) \\ &\quad \le \frac{1}{2}\sum_i (A_i^2 + S_i^2) + Q^\top A - Q^\top S. \end{aligned} \tag{24}$$

Proof. It is enough to prove the coordinate inequality. For $q, a, s \ge 0$, $([q - s]^+)^2 \le (q - s)^2$ and $[q - s]^+ \le q$. Hence

$$\begin{aligned} ([q - s]^+ + a)^2 &\le ([q - s]^+)^2 + a^2 + 2a[q - s]^+ \\ &\le q^2 + s^2 + a^2 + 2q(a - s). \end{aligned} \tag{25}$$

Subtracting q^2, dividing by two, and summing over coordinates gives the result.

THEOREM 1 **(Robust candidate MaxWeight stability certificate)**. *Consider a finite system with job classes $\mathcal{I}$ and queue dynamics* (1)*. Suppose Assumptions 1–4 hold. If*

$$\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1) > 0, \tag{26}$$

then for $V(Q) = \frac{1}{2}\sum_i Q_i^2$ and $\Delta V(t) = V(Q(t+1)) - V(Q(t))$,

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid Q(t) = Q] \le B + P_0 + \alpha_0 \\ - \eta \|Q\|_1 . \end{aligned} \tag{27}$$

Consequently, for any N satisfying

$$B + P_0 + \alpha_0 + \gamma \le \eta N \tag{28}$$

for some $\gamma > 0$, the process has a negative expected drift outside $\{Q : \|Q\|_1 \le N\}$. If Assumption 5 also holds, the standard countable-state Foster criterion gives positive recurrence of the corresponding closed communicating class.

Proof. Apply Lemma 3 with $(Q, A, S) = (Q(t), A(t), S(t))$ and condition on $Q(t) = Q$. Using Assumption 4,

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\le B + Q^\top \mathbb{E}[A(t) \mid Q(t) = Q] \\ &\quad - Q^\top \mathbb{E}[S(t) \mid Q(t) = Q] \\ &\le B + Q^\top \lambda - Q^\top \mu_t^{\mathrm{L}}(a_t).\end{aligned} \tag{29}$$

Write $D(Q) = Q^\top \lambda - Q^\top \mu_t^{\mathrm{L}}(a_t)$. Lemma 2 gives

$$\begin{aligned}D(Q) &\le P_0 + \alpha_0 \\ &\quad - \left(\delta - \epsilon_{\mathrm{cand}} - \epsilon_{\mathrm{est}} - \beta - \alpha_1\right) \|Q\|_1 .\end{aligned} \tag{30}$$

Combining the two displays proves

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\le B + P_0 + \alpha_0 \\ &\quad - \eta \|Q\|_1 .\end{aligned} \tag{31}$$

If $B + P_0 + \alpha_0 + \gamma \le \eta N$, the drift is at most $-\gamma$ whenever $\|Q\|_1 > N$. This is the finite-set Foster drift certificate. Under Assumption 5, the usual countable-state Foster theorem yields positive recurrence of the closed class.

COROLLARY 1 **(Statewise feasible families)**. *Let the candidate family and lower-service map be state indexed: $\mathcal{A}_t^{\mathrm{cand}} = \mathcal{A}^{\mathrm{cand}}(Q(t), \chi(t))$ and $\mu_t^{\mathrm{L}} = \mu^{\mathrm{L}}(Q(t), \chi(t), \cdot)$, where $\chi(t)$ records scheduler-visible availability, pinned jobs, node health, and measured capacity boundaries. If Assumptions 1–4 hold pointwise for every realized $(Q(t), \chi(t))$ with the same constants $\delta, \epsilon_{\mathrm{cand}}, \epsilon_{\mathrm{est}}, P_0, \beta, \alpha_0, \alpha_1, B$, then the drift bound in Theorem 1 holds unchanged.*

Proof. The proof of Lemmas 1–2 is pointwise in the realized candidate family and lower-service table. After conditioning on $Q(t) = Q$ and the scheduler-visible state $\chi(t)$, the same inequality chain gives the same lower-service margin and hence the same quadratic drift bound. Taking conditional expectation over $\chi(t)$ preserves the bound because all constants are uniform over the admitted statewise family.

The Lean 4 theorem prover artifact (de Moura and Ullrich 2021) proves the fixed-family theorem and the statewise version in Corollary 1, where the feasible family $\mathcal{A}_t^{\mathrm{cand}}$ depends on the queue state, available jobs, and measured capacity-boundary rows. It also proves a bounded finite-support specialization in which B is constructed from coordinate sample bounds. The paper theorem numbers are not Lean identifiers: the Lean crosswalk

maps Theorem 1 to `main_theorem_robust_candidate_maxweight_stability_from_calibrated_fabric_with_second_moment_bound_approx_oracle`, Corollary 1 to `main_statewise_calibrated_fabric_robust_candidate_stability_with_second_moment_bound_approx_oracle`, the coordinate-scaled lower confidence bound result to `main_diagonal_scaled_lcb_support_loss_l1`, and the operational sandwich boundary to `main_operational_capacity_sandwich`. *Claim boundary.* The theorem applies when the stated slack, lower-service, penalty, oracle-error, and moment assumptions are certified; it is not by itself a certificate for every scheduler dispatch.

4.3. Proof decomposition

The preceding lemmas isolate the mathematical obligations that must be certified by the scheduler. Lemma 1 is the capacity and candidate-approximation layer; Lemma 2 is the robust oracle layer; Lemma 3 is the queueing layer; and Theorem 1 is their Foster-Lyapunov composition. The Lean formalization mirrors this separation. The operational necessity direction is kept separate: an "operationally stabilizes" predicate is load-certified by a model that explicitly encodes the mean arrival vector, and zero-slack necessity follows from a conservation-law argument rather than from the definition of the predicate.

5. Scheduleurm Instantiation

5.1. Measured action slices

Scheduleurm is an existing scheduler for local and remote research jobs. The new algorithmic layer deliberately separates two interfaces. The theorem-facing interface builds global or statewise candidate families with `queue_vector`, `lower_service`, `penalty_units`, and `score_semantics=robust_maxweight_lower_service`; service profiling, replay, policy-semantics baselines, slack accounting, and oracle audits live under `simulation/` and `algorithm/experiments/`. The deployed live interface now has two opt-in hooks selected by policy name: a one-task placement scorer/admission gate and a global-batch soft-hint path for `global_theorem_maxweight_v1`. The latter builds a bounded global candidate action in `algorithm/` and passes only a temporary preferred node hint back to `scheduler.py`; final placement still goes through the legacy resource, claim, and launch checks. It is useful for A/B experiments, but it is not used in this paper as evidence that the production dispatcher already defaults to global robust MaxWeight dispatch.

Table 4 Current measured finite slices.

Bucket	Feasible profiles	Boundary	Candidate	Legacy
q00 `light_control_local`	1–13	14	13	1
q01 `gpu_heavy_jax_matmul`	1–8	none	1 replay; 4/8 cert.	3
q10 `cpu_heavy_local_bench`	1–9	10	8	9
q11 `hybrid_rl_resac_ant`	1–9	10	2	5

q01 uses profile 1 in online replay; profiles 4 and 8 remain measured support actions used by separate high-backlog support and lower confidence bound (LCB) certificates. q11 uses profile 2 in online replay and in the mixed portfolio. The q11 profile-10 replay point from earlier measurements is historical only: fresh live sanity showed runtime out-of-memory (OOM) at profile 10, making it the first robust capacity boundary for the current node bucket.

Table 4 summarizes the current measured finite slices. A capacity boundary is a measured or live sanity profile that is not usable as a robust feasible action. Once profile k is a boundary, profile k and all higher profiles are excluded from the theorem-facing candidate family for that workload bucket.

The replay policy uses a queue-adaptive robust MaxWeight penalty selector rather than a fixed co-location profile. For a workload with backlog count b, it first restricts profiles to a measured support envelope and then chooses profile k by the score

$$b\,\widehat{\mu}_i(k) - \theta_{\text{prof}}\widehat{\mu}_i^{\max}k, \qquad \theta_{\text{prof}} = 0.10, \tag{32}$$

where $\widehat{\mu}_i(k)$ is the measured aggregate service for class i at profile k, and $\widehat{\mu}_i^{\max}$ is the best measured aggregate service in the same envelope. Thus the replay candidate uses lower-interference profiles such as q01 profile 1 when they stay within the measured support slack, while support certificates retain actions such as q01 profiles 4 and 8 for capacity accounting. The optional statewise drain is lower-service guarded and no-upshift: it may only reduce a profile when the measured lower-service and waiting-backlog checks certify no loss relative to the base action. This implementation matches the approximate-oracle theorem: the support envelope preserves the drift action at large Q, and the finite-batch delay preference is a bounded profile penalty that is paid from the same slack budget as $K_t + G_t$.

Table 5 gives the implementation crosswalk. The purpose is to make the theorem-policy alignment auditable without pretending that every live dispatch call has already been upgraded to a global batch optimizer. When a row is generated by the theorem-facing pipeline, the robust MaxWeight objective is explicit. When a row is generated by the live hook, the row is an engineering action that must be enriched and audited before it is used as theorem evidence.

Table 5 Theorem-policy crosswalk in the Scheduleurm artifact.

Theorem object	Artifact field or module	Reviewer-facing interpretation
Queue vector Q	`queue_vector` in oracle traces and replay slots	Defines the backlog weights in $Q^\top \mu^{\mathrm{L}}(a)$.
Candidate action $a \in \mathcal{A}_t^{\mathrm{cand}}$	`action_model.py`, `adaptive_trace_policy.py`, and per-slot candidate rows	Represents a global or statewise placement/co-location configuration for the theorem-facing oracle.
Lower service $\mu^{\mathrm{L}}(a)$	`service_model.py` and measured service cache	Conservative service row used in the drift inequality.
Penalty $K_t(a) + G_t(a)$	`penalty_units`, `penalty_fit.py`, and bounded tie-break terms	Switching, risk, and delay-control terms that consume bounded or queue-scaled slack.
Oracle error (α_0, α_1)	`oracle_audit.py` and support-gap reports	Measures approximate maximization loss relative to the theorem objective.
Live placement hook	`placement.py` and `skill/scheduler.py` policy selector	Optional scheduler integration point; theorem use requires trace enrichment and oracle audit.
Submission closure runner	`or_submission_closure.py`	Runs online arrivals, holdout lower-service certificates, ablations, live-state trace audit, and Lean supplement repackaging without modifying `skill/scheduler.py`.

The theorem-facing rows expose robust MaxWeight score semantics explicitly; the live hook is an integration surface and not, by itself, a stability certificate.

5.2. Auditable theorem objects

The Scheduleurm artifact distinguishes three populations. The explicit task-list replay population is used for performance comparisons. The completed-active production population is used for a service-map theorem oracle bridge. The raw scheduler history is treated only as telemetry: it contains cancelled, attempted-only, benchmark, and externally adopted jobs and is not a clean arrival stream for the theorem.

The current completed-active Scheduleurm-controlled production population has 3437 mapped records out of 3437, positive mapped-slice capacity slack, and a service-map robust MaxWeight lower-service oracle bridge with $\alpha_0 = \alpha_1 = 0$. A separate live-state dry-run trace audit closes 128 synthetic placement slots and 767 candidate rows under the `sweetspot_v1` hook against the current probed node state. The dry-run probe does not write the queue and does not launch tasks. It proves the trace/enrichment/audit pipeline is executable on current scheduler candidate families at a longer candidate-slot scale; it is still a live-state dry-run audit, not statistical evidence for all online dispatches. A bounded launched q01/q11 ScheduleurmBench trace separately exercises the robust lower-service hook on real tasks and passes its completion bridge. The 2026-06-12 production queued trace then closes the current queued production population: one admissible BAPR task, one theorem slot, and an oracle audit pass, with no queue mutation and no launch/completion claim. A non-invasive active-production shadow trace copies current running production records into shadow queued records, evaluates the same optional theorem hook without mutating the queue, and closes the GPU theorem subset with $\alpha_0 = \alpha_1 = 0$. This shadow trace is useful evidence for live candidate

Table 6 Candidate policy versus legacy Scheduleurm on explicit task lists.

Task list	Candidate profile	Makespan	Mean flow	p90 flow
q00 light control	`light=13`	11.814×	11.758×	11.814×
q01 GPU compute	`gpu=1`	1.083×	1.149×	1.146×
q01 CNN ResNet-50	`cnn=3`	1.000×	1.000×	1.000×
q01 LLM DistilGPT-2	`llm=10`	2.801×	2.654×	2.673×
q01 GPU model portfolio	`gpu=1,cnn=3,llm=10`	1.172×	1.189×	1.194×
q10 CPU-host bound	`cpu=8`	1.539×	1.535×	1.534×
q11 CPU/GPU coupled	`hybrid=2`	1.175×	1.176×	1.156×
Mixed portfolio	`cpu=8,gpu=1,cnn=3,llm=10,hybrid=2`	1.174×	1.274×	1.203×

Ratios are legacy divided by candidate, so larger is better for Scheduleurm candidate. Each row uses the current robust feasible family; q11 profile 10 and above are excluded. p90 is the 90th percentile of job flow time.

semantics, but it is not a launched production dispatch trace. The raw 30-day telemetry window has 6957 records, of which 4825 are mapped and 2132 are unmapped, so its global coverage flag is intentionally false and it is not used as the theorem population.

6. Computational Evidence

6.1. Explicit task-list replay

The main benchmark uses one explicit task list per quadrant and one mixed portfolio. All policies receive the same jobs, total work units, resource-pool counts, arrival times, and measured service cache. Static arrivals are used for the first comparison because all-job completion time is then unambiguous. The online-arrival gate then runs Poisson and bursty traces at load factors 0.50, 0.70, 0.85, and 0.95 with three random seeds for each task set.

Table 6 compares the current candidate policy with the legacy Scheduleurm caps. The metric is completion of all jobs in the task list, not the time to finish a single job. In the task names, CNN denotes convolutional neural network and LLM denotes large language model.

Claim boundary. Table 6 is an all-job completion comparison on declared task lists and measured service-cache rows; it is not a claim about arbitrary future workloads.

Table 7 summarizes the online-arrival closure gate. It contains 192 scenarios: eight task sets, two arrival modes, four load factors, and three seeds. The adaptive candidate completed every job in every scenario. Relative to legacy caps, its geometric mean improvement was 1.670× for makespan and 4.998× for mean flow. The worst scenario-level makespan and mean-flow ratios over legacy were 1.004× and 0.999×, respectively; the latter is within the stated 0.5% replay tolerance and occurs in the ResNet-50 bucket where legacy and candidate use the same measured profile. The maximum observed candidate backlog was 90 jobs, and the maximum time-weighted mean backlog was 48.112 jobs.

Table 7 Online replay distribution over 192 scenarios.

Metric	Geomean/mean	Median	Min	5%	95%
Candidate / legacy makespan	1.670	1.329	1.004	1.020	9.835
Candidate / legacy mean flow	4.998	3.629	0.999	1.040	179.178
Candidate mean backlog jobs	7.398	8.300	0.941	1.659	27.757
Candidate max backlog jobs	23.493	25.000	4.000	6.000	67.000

The same artifact reports zero Pareto-dominated candidate scenarios and zero scenarios in which an ablated policy Pareto-dominated the full candidate. It also reports three single-metric losses beyond the 0.5% tolerance, all in one q01 LLM bursty high-load makespan row against table-goodput/delay/composite semantics; none is a Pareto-dominance violation.

The ablation gate separates the legacy caps, the scalar `sweetspot_v1` hook semantics, support-only makespan scoring, delay-only mean-flow scoring, statewise interference guarding, removal of the bounded profile penalty, and reversal of the profile tie-break. Across the 24 static, Poisson, and bursty traces, no ablated policy Pareto-dominated the full queue-adaptive robust lower-service scorer; the full candidate retained geometric improvements of 1.807× in makespan and 3.398× in mean flow over legacy. *Claim boundary.* These replay and ablation rows compare policy semantics on an identical measured service cache; they are not direct executions of external scheduler systems.

6.2. Policy-semantics replay baselines

No single external scheduler covers all four Scheduleurm quadrants and the mixed portfolio without changing the workload model. For this reason the external-scheduler evidence is organized as a diagnostic layer rather than as the central claim of the paper. We compare against policy-semantics replay baselines on the same measured service cache: a throughput-table goodput policy representing Gavel, Pollux, and Sia-like schedulers (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023); a finite-batch delay oracle representing shortest remaining processing time (SRPT), Gittins, and shortest expected remaining processing time (SERPT)-style policies (Scully et al. 2020); and an interference guard representing IADeep/Salus/Gandiva-like co-location control (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). Because every row is evaluated through the same measured lower-service map, these baselines test whether the finite candidate family is missing an obvious scheduling semantic; they are not direct binary executions of the external systems.

The same distinction is used for adapter artifacts. The registered-adapter gate maps 11 registered non-adjacent scheduler families into finite policy/service-unit rows in the Scheduleurm+external action union, and the strict measured-cache frontier is closed. This is the statement needed by the candidate-set theorem: after an additional finite action has a lower-service row and bounded penalty, adding it to the candidate family cannot harm the robust selector on the audited cache. Some

named external systems also have scoped same-host same-workload runtime audits, and Decima has a same-domain Spark directed acyclic graph (DAG) simulator execution audit. These artifacts are useful for traceability, but the paper does not pursue the open-ended claim that every external scheduler, every future workload, or every original multi-node control plane has a comparable end-to-end runtime certificate.

The aggregate claim gates therefore serve a narrow purpose: they make the boundary auditable. The universal-claim gate reports that the five scoped boundary certificates are executable, while only the strict scheduler-history production-completion row is a strong operational claim. The non-future gate reports that the finite named/external-adapter, registered policy-semantics, production-history, Scheduleurm-native multi-node, and Decima same-domain rows are closed after excluding arbitrary future workloads. This is a finite certificate ledger for the optimization model, not a systems benchmark competition.

The multi-node evidence is also strengthened without unsafe launching. A cross-node theorem-shadow gate probes the visible fabric and finds two reachable GPU nodes, `jtl110gpu` and `jtl110gpu2`. It constructs 18 certified lower-service candidate rows over q00/q01/q10/q11 synthetic workloads, enumerates 626 disjoint robust-MaxWeight configurations, and selects a configuration spanning both GPU nodes with $\alpha_0 = \alpha_1 = 0$. This closes the cross-node theorem-candidate interface, but not original multi-node full-stack superiority for external systems. A separate multi-node history gate audits strict theorem-admitted Scheduleurm production rows and finds 4,020 launched rows, 3,992 completed rows, zero strict unadmitted rows, 35 workload domains, 13 nodes, and three GPU nodes. This closes the Scheduleurm-native multi-node launched/completed history claim without claiming that every external SOTA control plane has been run in its original multi-node deployment.

Finally, the production organic boundary is moved from recorder-only readiness to strict-history completion evidence. The organic history completion gate audits real scheduler records, excludes raw-history rows outside scheduler control (attempted-only, auto-adopted, diagnostic, and read-only probe rows), and admits only exact/signature-certified service rows under strict theorem admission. It reports 4,020 strict organic launched rows, 3,992 completed rows, completion fraction 0.9930, 35 workload domains, 13 nodes, and zero strict unadmitted launched rows. The production-wide organic wrapper therefore reports `production_wide_history_completion_closed=true` and `large_scale_organic_launched_completion_ready=true`, while `production_wide_live_trace_closed=false` remains explicit. Thus the paper may claim

Table 8 Online policy-semantics replay aggregate on identical measured service cache.

Policy-semantics baseline	Candidate / makespan	Candidate / mean flow	Candidate / p90 flow
Throughput table goodput	1.0049×	1.0267×	1.0235×
Delay oracle	1.0049×	1.0267×	1.0235×
Interference guard	1.0009×	1.0032×	1.0029×
Quadrant composite	1.0049×	1.0267×	1.0235×

Entries are geometric mean candidate-versus-policy ratios across the online arrival scenarios. Values above one favor the candidate. A row would dominate the candidate only if both makespan and mean-flow ratios were materially below one; no such row appears under the 0.5% replay tolerance.

strict scheduler-history organic launched/completion evidence, not raw history, future unknown arrivals, or a statement that every live slot has an emitted oracle-trace row.

Table 8 reports the online policy-semantics aggregate over the 192 Poisson and bursty load-sweep scenarios. The candidate is not Pareto-dominated by any external-policy diagnostic row under the same measured service cache. The expanded q01 CNN/LLM buckets remove the earlier aggregate makespan tradeoff with the throughput-table endpoint: the candidate now improves geometric-mean makespan and flow against that endpoint, although individual scenarios remain within the stated replay tolerance.

We also evaluate the stronger theory-facing construction in which external policy-family actions are not merely baselines but are included in the candidate set. The selector evaluates the Scheduleurm candidate action together with the throughput-table, delay-oracle, interference-guard, finish-time-fairness, resource-adaptive, packing-guard, and measured bridge-action families under the same measured service units. The action is not deduplicated by co-location profile alone: a candidate is a finite configuration trajectory containing the placement/profile, resource partition, admission order, and statewise drain rule. This distinction matters on the ResNet-50 bucket, where the same profile supports different finish-time and interference-drain trajectories. Table 9 shows that the metric-specific action unions match or improve the external-policy envelopes on every measured task-list scenario within the 0.5% replay tolerance. The fixed Pareto-slack union policy goes one step further: for each visible queue portfolio it enumerates the finite measured Scheduleurm+external action family and chooses the configuration maximizing the smaller of the normalized makespan and mean-flow slack. This single online rule is not Pareto-dominated by any external-policy diagnostic row and simultaneously closes both external-policy envelopes on the measured cache within tolerance; the guarded and adaptive-scalarized rows remain useful ablations rather than the main dominance certificate.

We keep the strict measured-cache claim separate from any direct-system claim. The final measured-cache external-policy frontier gate reports the literal artifact flag `MEASURED_CACHE_`

Table 9 External-policy action-union gate on identical measured service cache.

Policy family	Sum makespan	Mean flow	Gate meaning
Current Scheduleurm candidate	153168.631	3171.800	Baseline candidate.
External best makespan fixed policy	152822.454	3171.388	Gavel/Themis-style finish-time fairness.
External best flow fixed policy	152823.045	3171.388	IADeep/Salus-style interference guard.
Scheduleurm+external guarded union	152822.320	3171.385	Not Pareto-dominated by any external row.
Scheduleurm+external Pareto-slack union	152822.320	3171.385	Single fixed online policy closes both external-policy envelopes.
Scheduleurm+external mean-flow union	152822.320	3171.385	Metric-specific mean-flow envelope certificate.

Times are in seconds. The action-union rows are policy-semantics selectors over measured service actions, not direct executions of Gavel, Pollux, Sia, IADeep, or Salus. The Pareto-slack policy is not Pareto-dominated by any external-policy diagnostic row and closes both measured-cache external-policy envelopes; the measured-cache external-policy frontier artifact reports zero remaining below-envelope rows.

`EXTERNAL_POLICY_FRONTIER_CLOSED`, meaning that every measured-cache scenario is strictly noninferior to the external-policy envelope in both makespan and mean flow. This flag is not a direct full-stack superiority claim over Gavel, Pollux, Sia, IADeep, or Salus; that stronger claim is governed by the separate full-stack gate. The closing action is a finite ResNet-50 trajectory bridge, not a new service rate: for static queues it applies a deterministic hash partition with seed 30 across the two GPUs, runs the measured profile-3 action, and drains the last two remaining jobs with measured profile 1. This closes the q01 ResNet-50 row, the q01 model portfolio row, and the mixed-portfolio inherited row. The four-quadrant gate therefore passes strict noninferiority for q00, q01, q10, q11, and the mixed portfolio. It still does not certify strict Pareto dominance in every quadrant: q00, q10, and q11 are exact ties against the best external-policy envelope, while q01 and the mixed portfolio contain strict improvements.

The follow-on external-policy algorithm-upgrade gate closes the corresponding implementation layer without changing the default production scheduler. It adds the fixed Pareto-slack Scheduleurm+external union policy, the adaptive scalarized ablation, a state-dependent marginal service cache, bounded global batch lookahead, reusable online LCB estimated time remaining (ETA) estimates, and three additional external-policy action families: finish-time fairness, Sia/Pollux-style resource-adaptive goodput, and Salus/IADeep/Gandiva-style packing guards. The Pareto-slack policy closes the measured-cache policy-semantics external envelope; the metric-specific union rows remain diagnostic envelope certificates. The marginal-service rows explicitly record the observed non-monotonic opportunities: high video random-access memory (VRAM) resident small CUDA

(Compute Unified Device Architecture) service is 1.095× the empty reference in the controlled slice, and CPU-resident marginal rows are 1.001×–1.091× the empty CPU reference. These additions strengthen the finite-action candidate-set claim; they remain replay/certificate evidence, not launched production default dispatch or direct full-stack superiority over external systems.

On the q01 GPU-heavy list, the node007 profile-1 measurement now lies on the measured service envelope, so the candidate and the table-goodput/delay external-policy rows all select profile 1 on the single-workload q01 replay. The implemented statewise logic is a lower-service no-upshift drain, not a hidden post-hoc rule: it cannot raise the base action and it only lowers a profile after service and waiting-backlog guards pass. The 2026-06-13 optional algorithm-upgrade gate adds a global batch candidate constructor, load-state marginal service lookup, and online LCB/ETA updater on top of this replay policy. In its scoped replay certificate, q01 compute, q01 LLM, q01 model-portfolio, and the mixed portfolio improve relative to the current candidate, while q01 CNN, q10, and q11 remain non-regressed under the 0.5% replay tolerance. This is an opt-in algorithm-layer certificate; it is not evidence that production dispatches already use global batch launch by default.

6.3. Theorem-condition and live-sanity checks

The measured portfolio finite slice also instantiates the slack inequality in Theorem 1. With load $\lambda_i = 0.8\,\mu_i(a_{\text{selected}})$, the candidate family equals the measured action slice, so $L\rho = 0$. The resulting certificate is

$$\begin{aligned} \delta &= 0.078777710, \\ \epsilon_{\text{est}} = \beta = \alpha_1 &= 0, \\ \eta &= 0.078777710. \end{aligned} \tag{33}$$

The finite-support second-moment bound is $B = 34499765.4854296$, and the resulting finite-set threshold is $N = 437938174$. This certificate enumerates 5832 measured actions after adding the ResNet-50 and DistilGPT-2 q01 profiles to the mixed portfolio. This is a theorem-condition certificate for a declared finite-support load model; it is not a claim that the static replay trace is itself a stationary arrival process. The zero modeled losses arise because this certificate uses an exact enumerated measured slice and an exact service-map oracle. They should not be read as evidence that a broad fabric-cover metric has already been calibrated with $L\rho = 0$, or that holdout lower-service error vanishes outside the measured rows. For smaller candidate families, the artifact

Table 10 Finite measured lower-service capacity certificates.

Task set	High-backlog selected profiles	η
q00 light control	`light=13`	123.0121
q01 GPU-bound compute	`gpu=4`	15.1785
q01 CNN ResNet-50	`cnn=3`	20.4221
q01 LLM DistilGPT-2	`llm=10`	1296.9301
q01 GPU model portfolio	`gpu=8,cnn=3,llm=10`	14.2529
q10 CPU-host bound	`cpu=8`	10.9559
q11 CPU/GPU coupled	`hybrid=2`	0.067943
Mixed portfolio	`cpu=8,gpu=1,cnn=3,llm=10,hybrid=3`	0.066922

Each row is a stability certificate, not a replay-performance estimate: the finite-slice rows use the finite measured lower-service model at load fraction 0.8, and the selected-profile stochastic row uses the LCB lower-service model described in the text. Performance replay tables elsewhere use empirical measured means on the same service cache. The certificates here support the declared finite slice and do not claim stochastic generalization beyond the measured or LCB-admitted profiles.

now reports a greedy finite-feature k-center cover curve; those rows quantify the nonzero $L\rho$ that must be paid before invoking the fabric-cover theorem away from the exact measured slice.

The Operations Research (OR) closure runner also computes a selected-profile stochastic LCB diagnostic. The diagnostic is now computed on profile-level aggregate service windows rather than individual co-located task rates, matching the service coordinate used by the theorem. All 11 selected targets meet the 20 aggregate-window threshold after adding 921 supplemental raw progress rates. Two stronger mean-service claims are still false and are reported separately: the absolute mean-service holdout diagnostic has $\epsilon_{\text{est}} = 739.0608$, giving $\eta = -738.9820$, and the diagonal-normalized mean-service diagnostic has $\epsilon_{\text{est}}^{\text{norm}} = 0.7587$ against normalized slack $\delta^{\text{norm}} = 0.2$, giving $\eta^{\text{norm}} = -0.5587$. The theorem-facing stochastic claim therefore uses the LCB service itself as the lower-service action family. For arrivals scaled to 0.8 of this certified LCB service vector, the selected-profile lower-service capacity certificate has positive slack $\delta_{\text{LCB}} = 0.024712$. Its coordinate LCB ratios for CPU, CNN, JAX GPU, LLM, and hybrid RL are 0.3560, 0.5843, 0.7881, 0.8769, and 0.3137, respectively. This is a stochastic lower-service capacity claim, not a claim that 0.8 of the measured mean-service vector is certified. Table 10 reports the resulting positive drift margins. *Claim boundary.* The positive η values in Table 10 are finite measured-slice certificates. The selected-profile stochastic LCB gate is now passed only in the narrower lower-service capacity sense above; the absolute and diagonal-normalized mean-service eta gates remain false.

The replay model was checked against fresh progress-window live measurements on representative measured service-cache rows, not only on the final selected profiles. For the q01 profile-4 row, replay predicted a makespan of 1661.313 seconds, whereas the fresh live proxy was 1652.248 seconds, a relative error of 0.005457. For q11 profile 2, replay predicted 38382.365 seconds and

Table 11 Controlled corner-case marginal service probes.

Scenario	Node	Resident rate	Marginal rate	Loaded/empty ratio
CPU half resident + small CPU	`node001`	12.413	22.312	1.090
CPU full resident + small CPU	`node001`	7.888	20.470	1.000
28.8GB GPU resident + small CUDA	`node007-direct`	2359.103	8325.145	0.975
30.0GB GPU resident + small CUDA	`node007-direct`	2342.603	9367.452	1.098

Rates are benchmark service units per second from explicit progress logs. The loaded/empty ratio compares the marginal row with the same small-task empty-resource baseline. The table is a controlled synthetic marginal-service slice, not a full-stack SOTA performance claim or a production-wide theorem population.

the fresh live proxy was 37760.000 seconds, a relative error of 0.016215. For the mixed portfolio, composed live slices gave relative errors of 0.007349 on makespan, 0.005417 on mean flow, and 0.007964 on p90 flow. These are progress-window sanity checks; the long jobs were not all run to natural completion.

We also ran a corner-case marginal-efficiency gate while the cluster was idle. Each probe used benchmark logs with explicit progress rate and estimated time remaining (ETA) lines; after three stable windows the probe stopped and the measured service row was available for replay. Table 11 reports the last stable marginal rate and the ratio to the corresponding empty resource baseline. The 30GB row holds a four-GPU resident memory job on `node007-direct` and then adds a small CUDA task on one card. The CPU rows run 96-worker and 180-worker resident jobs on 192-core nodes and then add a small CPU task. These rows show that high resident video memory does not by itself imply a low marginal CUDA rate on this slice; on the idle 192-core CPU node, the single-thread marginal CPU micro-kernel also retained its empty-node rate under the 96-worker and 180-worker resident loads used here. The gate also records a policy admission matrix: Scheduleurm's theorem hook admits all stable positive-service rows; Salus/Gandiva-style memory packing admits the GPU rows when memory fits; Pollux/Sia-style goodput admits the rows whose loaded-to-empty ratio exceeds 0.90. The corner-case lower-service gate then admits the five stable canonical rows into a standalone service-cache snapshot and verifies positive row-level capacity slack under an offered load of 0.8 times the measured lower-service rate. This makes the corner-case rows theorem-facing for this finite measured slice without changing the default replay cache. Table 12 then reports the same rows using job completion time (JCT) rather than marginal service rate.

A separate launched live theorem-dispatch gate ran controlled q01/q11 Scheduleurm-Bench tasks through the optional robust lower-service hook. The reviewer-facing completion artifacts report four flags rather than one collapsed production claim. First, the bounded controlled run (`md/controlled_production_completion_gate_20260612.md`)

Table 12 Gavel-style resident-delay/JCT holdout on the controlled co-location rows.

Scenario	Co-run mean JCT	Defer mean JCT	Mean JCT reduction	Makespan reduction
CPU half resident + small CPU	0.752	0.988	23.9%	24.5%
CPU full resident + small CPU	1.054	1.464	28.0%	13.4%
30.0GB GPU resident + small CUDA	0.029	0.054	46.7%	11.0%

Seconds are measured benchmark service-window seconds. The defer baseline uses the fastest observed resident-alone progress window, which favors defer. The table is a scoped measured-service holdout, not a direct Gavel full-stack execution.

has `controlled_6_task_completion_ready=true`: it launched 6 real tasks, emitted 7 theorem-grade dispatch slots and 13 candidate rows, and passed the completion bridge; one remote launch failed because the working directory was unavailable and was retried on a certified local candidate. Second, the controlled 32-task ScheduleurmBench run (`md/controlled_jtl110gpu_32_20260613.md`) has `controlled_32_task_completion_ready=true`: it launched 32 theorem-admitted tasks on an available two-GPU node, emitted 32 robust lower-service theorem slots and 56 candidate rows, and completed all 32 tasks with $\alpha_0 = \alpha_1 = 0$. These controlled runs are live launched sanity checks, not production-wide traces.

The rolling production evidence is separated in the same way. The strict scheduler-history gate (`md/organic_history_completion_gate_20260614.md`) reports `strict_history_large_scale_completion_ready=true`: 4,020 strict theorem-facing organic launches, 3,992 completions, 35 admitted workload domains, 13 nodes, and zero strict unadmitted launched rows. The production-launch wrapper (`md/production_launch_completion_gate_20260612.md`) remains rerunnable on rolling queue snapshots and reports active-production progress and non-invasive shadow trace separately from this strict history counter. Its live oracle-traced large-scale completion flag is `live_oracle_traced_large_scale_completion_ready=false`; this is intentional because the large-scale production evidence is scheduler-history completion, not a large launched live-oracle trace. For closed shadow-subset snapshots, the theorem bridge is usable with $\alpha_0 = \alpha_1 = 0$; CPU fallback slots use legacy scheduler sort-key semantics and are deliberately excluded from the theorem subset. A read-only multi-node inventory sees two reachable GPU nodes, `jtl110gpu` and `jtl110gpu2`; the Scheduleurm-native multi-node history gate is closed, but external systems have still not been run through their native multi-node control-plane and worker paths on a shared workload. *Claim boundary.* The live evidence certifies bounded controlled completion, non-invasive production shadow semantics, strict scheduler-history organic completion, and

Scheduleurm-native multi-node history. It does not certify raw history, future unknown workloads, every live oracle slot, a large-scale live oracle-traced production completion trace, or external SOTA original multi-node superiority.

6.4. Formal and production artifacts

The Lean proof artifact is submitted separately as a consolidated file `ScheduleurmUpload.lean`. The checked artifact has Secure Hash Algorithm 256-bit (SHA-256) hash

`25f0c744fb10fe8a9c3399f5072d5d16a81084082ab0264e92e9e7c8d08739c2.`

The commands `lake build Scheduleurm` and `lake env lean ScheduleurmUpload.lean` both pass. The current reviewer supplement rerun copied `ScheduleurmUpload.lean`, `lakefile.toml`, `lean-toolchain`, the paper-theorem to Lean-theorem crosswalk, and a path-independent build log. A comment-aware search for `sorry`, `admit`, and `axiom` returns no proof-term matches. The theorem names used in this paper are searchable in the upload file; the hash above is meaningful only together with the supplement manifest and build log.

For production data, the current theorem-facing population is the completed-active Scheduleurm-controlled set. It has 3437 strict mapped records out of 3437, with mapped-slice slack 9.123×10^{-6}. The service-map oracle bridge passes with $\alpha_0 = \alpha_1 = 0$. By contrast, the raw 30-day history has 6957 records, with 4825 mapped records and 2132 unmapped records, so its mapped fraction is about 0.694 and its global theorem-coverage flag is false. This distinction is important: the artifact proves coverage of a controlled completed-active population, not closure of every raw, attempted-only, external, or future scheduler row.

7. Related Work

7.1. Deep-learning and heterogeneous cluster schedulers

Goodput-oriented deep-learning schedulers such as Gavel, Pollux, and Sia use measured or modeled throughput tables to allocate heterogeneous accelerators (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023). Fairness-oriented systems such as Themis study efficient sharing under fairness constraints (Mahajan et al. 2020). GPU sharing systems such as Gandiva, Salus, and IADeep expose or exploit fine-grained co-location and interference behavior (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). These systems motivate our measured-service replay baselines, but our theoretical question is different: we ask how a scheduler can restrict a combinatorial configuration action space to a candidate family while preserving a queueing-stability certificate with explicit loss terms.

Hybrid CPU/GPU placement and cluster allocation systems such as AlloX and related heterogeneous placement work also emphasize that accelerator scheduling cannot be reduced to GPU count alone (Le et al. 2020, Carvalho et al. 2024). Scheduleurm follows the same empirical lesson. Its q00, q10, and q11 buckets are declared local or node-bucket service certificates, not claims that all CPU-heavy or RL workloads have the same service curve.

7.2. Queueing-network control and learning

MaxWeight and backpressure policies are classical tools for stabilizing constrained queueing systems under capacity-region slack (Tassiulas and Ephremides 1992, Stolyar 2004). Foster-Lyapunov and fluid-limit arguments also underlie positive recurrence and moment control in multiclass queueing networks (Dai and Meyn 1995). Our proof follows this lineage but adds three implementation-facing terms: candidate-set approximation, lower-service estimation, and approximate-oracle loss. Work on state-dependent processor sharing and workload-dependent admission control studies service rates that depend on the number or composition of active jobs (Gupta and Zhang 2022, Bekker and Borst 2006, Altman et al. 2006). The Scheduleurm co-location curves are a systems instance of the same phenomenon.

Learning-while-scheduling work, including MaxWeight with discounted upper confidence bound (UCB) and learning-based queueing controls, studies unknown service statistics under online exploration (Yang et al. 2023, Liu et al. 2022, Dai and Gluzman 2022). We do not claim a complete online learning theorem for the current Scheduleurm sampler. The formal artifact proves conditional high-probability lifting for active-bucket certificates. The current experiment artifact closes the deterministic finite active-bucket union bound and replay dwell/switching accounting, and adds a concrete deployable probability model: deterministic round-robin forced exploration over the finite active bucket set and a bounded two-window mean-shift detector. The certificate covers 64 active buckets and 120 replay scenarios with minimum forced samples 1561 per bucket, Hoeffding radius 0.0523, detector union tail bound 0.00564, and detection-delay bound 1024 decisions. A 2026-06-12 optional live-state probe further verifies that `adaptive_theorem_maxweight_v1` emits active-bucket sampler and regime-detector audit fields while preserving the lower-service theorem oracle. This remains an opt-in extension path; the default live scheduler policy is not changed.

8. Discussion and Limitations

The main contribution of this work is a bridge between Operations Research (OR)-style queueing certificates and a concrete scheduler artifact. The theory does not shrink the full action space into

the implemented candidate set. Instead, it states the loss incurred by that restriction and pays it from capacity slack. The Scheduleurm implementation then makes the proof objects auditable: measured lower-service rows, capacity boundaries, explicit task lists, finite-slice slack accounting, and lower-service oracle traces are all materialized as artifacts.

Several limitations remain. First, the strongest empirical claim is an exact measured finite-slice claim. We now calibrate the finite-feature metric and the service-sensitivity envelope on those measured slices, and the main measured certificate uses the exact candidate family with $\rho = 0$. The greedy k-center cover curve reports how $L\rho$ changes as the candidate family is smaller than the measured full slice, but a broader positive-service future/all-state claim would still require perturbation evidence for the claimed state population, projection π, cover radius ρ, service envelope L, and failure probability if the envelope is statistical; the current artifact includes a metric-contract table for the measured finite population, including the feature map, numeric scales, projection population, exact ρ, compressed-cover $L\rho$, and exclusions. The future-admitted fabric-cover gate closes a restricted future claim by construction: if a future task is admitted to one of 120 exact measured measured workload domains, the theorem-facing projection is identity and $\rho = 0$; otherwise the task is probe-required and excluded from the positive-load theorem-facing stream. A declared finite-domain positive-cover gate formalizes that population as 225 exact service-cache buckets: 215 have positive lower service, 10 are measured capacity boundaries, and zero are uncovered, so the declared-domain classification fraction is 1.0, the positive-service fraction is $215/225$, and the boundary fraction is $10/225$. A separate conservative all-state safety gate covers every scheduler-visible state by partitioning it into measured-admitted states or an unknown probe/defer action with zero theorem service. This closes the declared finite positive domain and all-state safety, not arbitrary positive-service all-state stability. Second, the external-scheduler material is deliberately supplemental. The policy-semantics comparisons replay finite goodput, delay, and interference rules on the same measured Scheduleurm service cache; they are diagnostics for whether the candidate action family has omitted an important scheduling semantic. The registered adapter gate then converts the same idea into the language of the theorem: each admitted external family is just another finite action with a lower-service row and bounded penalty. Native or same-host runtime audits for named systems are kept in the artifact package for traceability, but the manuscript does not rely on a universal full-stack SOTA superiority statement. Third, the live oracle trace evidence separates dry-run candidate-family audits, bounded launched Scheduleurm-Bench dispatch, read-only current queued-production trace, and non-invasive active-production

shadow probing. The bounded launched trace validates the hook on controlled tasks, the queued-production trace validates the current queued population without launch, and the shadow trace validates current active-production GPU theorem subset semantics without queue mutation. A future-production admission contract now routes active or queued production jobs through the literal artifact states `ADMIT_THEOREM_TRACE` or `PROBE_REQUIRED`. The production launch/completion gate then enforces the operational side: 2026-06-12 gate snapshots observed dozens of active production jobs and refused to promote launched-completion claims unless the theorem-shadow and completion thresholds were met. The controlled launched-completion gate now selects a 32-task controlled ScheduleurmBench run with 32 completed launched tasks, 32 theorem slots, 56 candidate rows, and $\alpha_0 = \alpha_1 = 0$. This closes controlled launched completion while keeping large-scale organic production completion false. A separate selected-profile holdout LCB gate tracks the stochastic-generalization upgrade: all selected aggregate-window sample thresholds are now met and the LCB lower-service capacity gate passes with positive slack, while the absolute and diagonal mean-service eta gates remain negative. Thus the claim is stochastic lower-service capacity, not 0.8 mean-service certification.

Fourth, q00 and q10 now have a broad measured-envelope gate over the service cache: one q00 light-control workload and 102 q10 CPU-like workloads have positive measured lower-service rows. This supports a measured-envelope claim for admitted q00/q10 classes, but not a universal claim over every CPU-heavy or data-loader-heavy program. The q11 result remains a declared robust node-bucket certificate and should not be generalized to all RL/BAPR deployments without replication on those buckets. The controlled corner-case service-cache gate now admits the node007/node001 resident-load rows into a standalone cache and verifies row-level lower-service slack, but it is still a finite measured slice, not an all-state production theorem.

These limitations are not incidental. They are the boundary that makes the claims falsifiable and therefore stronger. The next empirical step is to run larger launched production-wide theorem traces when queued production jobs and low-interference resources are simultaneously available. External Kubernetes/Docker/Go/runtime deployments can be added as supplementary adapter audits, but they are not required for the paper's optimization theorem. The next theoretical-to-systems step is to A/B validate the optional active-bucket sampler and bounded two-window detector before promoting them from extension certificates to the default scheduler policy.

9. Conclusion

We formulated heterogeneous compute scheduling as a configuration-action stochastic processing network and proved a robust candidate MaxWeight stability certificate with explicit slack accounting. The proof identifies precisely how candidate approximation, lower-service estimation, penalties, and approximate optimization consume capacity slack. In Scheduleurm, measured finite service slices, explicit task-list replay, online arrival stress replay, finite lower-service capacity certificates, production coverage, and live-state dry-run oracle-trace audits instantiate the theorem objects under a bounded claim scope. The current evidence supports a mathematical conclusion rather than a universal engineering conclusion: for declared finite action families with measured lower-service rows and audited slack, the robust candidate rule has a closed stability certificate, and the Scheduleurm case study shows how a real scheduler can produce the objects required by that certificate. Policy replay and adapter audits are included to check the finite candidate model, not to claim exhaustive superiority over external scheduler systems.

Appendix: Conventional Proof Details

This appendix gives the conventional, paper-readable proof chain behind Theorem 1. It is intentionally redundant with the main text: the purpose is to make clear which part is mathematics, which part is a measured Scheduleurm certificate, and which part is checked by Lean. The Lean artifact is a formal audit of the algebraic and recurrence implications; it is not a substitute for empirical certificates of service rows, lower confidence bounds, candidate admission, oracle gaps, or moment bounds.

EC.1. Capacity Slack as a Support-Function Margin

Let $M(a) \in \mathbb{R}_+^{|\mathcal{I}|}$ denote the true mean service vector of full action $a \in \mathcal{A}^{\text{full}}$, and let $H_{\mathcal{A}^{\text{full}}}(q) = \max_{a \in \mathcal{A}^{\text{full}}} q^\top M(a)$ be the full-action support function for nonnegative backlog weights $q \in \mathbb{R}_+^{|\mathcal{I}|}$. A stationary randomized policy over full actions is a probability vector $x \in \Delta(\mathcal{A}^{\text{full}})$. If it has coordinate slack $\delta > 0$ against the arrival mean λ, then

$$\lambda_i + \delta \le \sum_{a \in \mathcal{A}^{\text{full}}} x_a M_i(a), \qquad i \in \mathcal{I}. \tag{34}$$

Multiplying each coordinate by $q_i \ge 0$ and summing gives

$$q^\top \lambda + \delta \, \|q\|_1 \le \sum_{a \in \mathcal{A}^{\text{full}}} x_a q^\top M(a). \tag{35}$$

Because x is a convex combination, the right-hand side is at most the maximum full-action support:

$$\sum_{a\in\mathcal{A}^{\text{full}}} x_a q^\top M(a) \le H_{\mathcal{A}^{\text{full}}}(q). \tag{36}$$

Hence full-action capacity slack implies

$$q^\top \lambda + \delta \|q\|_1 \le H_{\mathcal{A}^{\text{full}}}(q). \tag{37}$$

This is the only place where the capacity-region hypothesis enters the drift proof. It converts an interior capacity statement into a MaxWeight support margin that can be compared to candidate actions. The paper uses the support-function statement as Assumption 1; the operational necessity direction is separate and requires a load-certified stochastic model and a conservation law.

EC.2. Candidate Fabric Cover and Support Loss

The scheduler cannot enumerate every full configuration. It evaluates an admitted candidate family $\mathcal{A}_t^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$, possibly depending on the observed state t. The finite-candidate theorem needs a support loss certificate: for every $q \ge 0$,

$$H_{\mathcal{A}^{\text{full}}}(q) \le H_{\mathcal{A}_t^{\text{cand}}}(q) + \epsilon_{\text{cand}} \|q\|_1 . \tag{38}$$

When the loss is obtained from a calibrated fabric metric, let $\Phi(a)$ be the finite feature representation of action a, let d be the feature metric, and suppose each full action a has a projected candidate $\pi_t(a) \in \mathcal{A}_t^{\text{cand}}$ satisfying

$$d(\Phi(a), \Phi(\pi_t(a))) \le \rho. \tag{39}$$

If service is Lipschitz in this representation,

$$\|M(a) - M(\pi_t(a))\|_\infty \le L\, d(\Phi(a), \Phi(\pi_t(a))), \tag{40}$$

then for any maximizer $a^\star \in \arg\max_{a\in\mathcal{A}^{\text{full}}} q^\top M(a)$,

$$\begin{aligned} H_{\mathcal{A}^{\text{full}}}(q) &= q^\top M(a^\star) \\ &\le q^\top M(\pi_t(a^\star)) + \|q\|_1 L\rho \\ &\le H_{\mathcal{A}_t^{\text{cand}}}(q) + L\rho \|q\|_1 . \end{aligned} \tag{41}$$

Thus $\epsilon_{\text{cand}} = L\rho$ in the fabric-cover case. In measured finite slices where the candidate support is certified directly, the artifact uses the observed support gap itself as ϵ_{cand}; when the candidate set is exact on the measured slice, this term is zero. This is why the proof does not shrink the action space informally: every restriction must be paid for by an explicit support-function loss.

EC.3. Lower-Service Rows, Penalties, and Approximate Oracles

The robust scheduler optimizes conservative service, not the unknown true mean service. For each admitted candidate $a \in \mathcal{A}_t^{\mathrm{cand}}$, the artifact provides a lower-service vector $\mu_t^{\mathrm{L}}(a)$. The lower-service support loss is

$$H_{\mathcal{A}_t^{\mathrm{cand}}}(q) \le \max_{a \in \mathcal{A}_t^{\mathrm{cand}}} q^\top \mu_t^{\mathrm{L}}(a) + \epsilon_{\mathrm{est}} \|q\|_1 . \tag{42}$$

The scheduling score is

$$\Psi_t(a) = Q^\top \mu_t^{\mathrm{L}}(a) - K_t(a) - G_t(a), \tag{43}$$

where $K_t(a)$ and $G_t(a)$ are bounded switching, risk, or guard penalties. The selected action a_t need not be the exact maximizer. It satisfies

$$\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} \Psi_t(a) - \Psi_t(a_t) \le \alpha_0 + \alpha_1 \|Q\|_1 , \tag{44}$$

and the selected penalty obeys

$$0 \le K_t(a_t) + G_t(a_t) \le P_0 + \beta \|Q\|_1 . \tag{45}$$

Combining (37)–(45) yields the selected lower-service margin. First,

$$\max_{a \in \mathcal{A}_t^{\mathrm{cand}}} Q^\top \mu_t^{\mathrm{L}}(a) \ge Q^\top \lambda + (\delta - \epsilon_{\mathrm{cand}} - \epsilon_{\mathrm{est}}) \|Q\|_1 . \tag{46}$$

Second, by the oracle gap and the selected penalty bound,

$$\begin{aligned} Q^\top \mu_t^{\mathrm{L}}(a_t) &\ge \max_{a \in \mathcal{A}_t^{\mathrm{cand}}} Q^\top \mu_t^{\mathrm{L}}(a) \\ &\quad - (P_0 + \alpha_0) - (\beta + \alpha_1) \|Q\|_1 \\ &\ge Q^\top \lambda + \eta \|Q\|_1 - (P_0 + \alpha_0), \end{aligned} \tag{47}$$

where

$$\eta = \delta - (\epsilon_{\mathrm{cand}} + \epsilon_{\mathrm{est}} + \beta + \alpha_1). \tag{48}$$

The theorem therefore has a simple accounting rule: candidate loss, lower-service estimation loss, queue-scaled penalty, and queue-scaled oracle loss all consume the same capacity slack. A stability certificate requires $\eta > 0$. Additive terms P_0 and α_0 do not change the slope of the drift; they enlarge the finite set outside which the drift is negative.

EC.4. Quadratic Drift and the Foster Certificate

The queue update is

$$Q_i(t+1) = \left[Q_i(t) - S_i(t)\right]^+ + A_i(t), \tag{49}$$

where $A_i(t)$ is arrival and $S_i(t)$ is realized service for class i. For $V(Q) = \frac{1}{2}\sum_i Q_i^2$, the standard one-step inequality gives

$$\begin{aligned}\Delta V(t) \le \frac{1}{2}\sum_i \left(A_i(t)^2 + S_i(t)^2\right) \\ + Q(t)^\top A(t) - Q(t)^\top S(t).\end{aligned} \tag{50}$$

Assumption 4 supplies the conditional second-order bound

$$\mathbb{E}\left[\frac{1}{2}\sum_i \left(A_i(t)^2 + S_i(t)^2\right) \,\middle|\, Q(t) = Q\right] \le B. \tag{51}$$

It also ties the stochastic process to the theorem-facing service rows:

$$\begin{aligned}\mathbb{E}[A(t) \mid Q(t) = Q] \le \lambda, \\ \mathbb{E}[S(t) \mid Q(t) = Q] \ge \mu_t^{\mathrm{L}}(a_t)\end{aligned} \tag{52}$$

coordinatewise. Taking conditional expectation in (50) and using (47),

$$\begin{aligned}\mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\le B + Q^\top \lambda - Q^\top \mu_t^{\mathrm{L}}(a_t) \\ &\le B + P_0 + \alpha_0 - \eta \left\|Q\right\|_1 .\end{aligned} \tag{53}$$

If N and $\gamma > 0$ satisfy

$$B + P_0 + \alpha_0 + \gamma \le \eta N, \tag{54}$$

then whenever $\|Q\|_1 > N$, (53) implies

$$\mathbb{E}[\Delta V(t) \mid Q(t) = Q] \le -\gamma. \tag{55}$$

This is the finite-set drift certificate used in Theorem 1. To translate the certificate into positive recurrence for a Markov chain, the paper also assumes a countable queue state space, finite sublevel sets of V, and a closed communicating class or equivalent local-return condition. Without those recurrence-side conditions the theorem should be read as a negative-drift certificate, not as an unconditional positive-recurrence statement for every possible implementation.

Table 13 Paper theorem to Lean theorem crosswalk.

Paper statement	Lean identifier	What the Lean theorem checks
Theorem 1, exact support-cover version	`main_theorem_robust_candidate_maxweight_stability_under_fabric_cover`	Exact-oracle support loss, robust drift, and recurrence certificate.
Theorem 1, approximate-oracle version	`main_theorem_robust_candidate_maxweight_stability_from_calibrated_fabric_with_second_moment_bound_approx_oracle`	Calibrated cover, bounded second moment, approximate oracle, and slack accounting.
Corollary 1	`main_statewise_calibrated_fabric_robust_candidate_stability_with_second_moment_bound_approx_oracle`	State-dependent candidate families with uniform constants.
Coordinate-scaled lower-service certificate	`main_diagonal_scaled_lcb_support_loss_l1`	Heterogeneous service-unit scaling and lower confidence bound support loss.
Operational capacity boundary	`main_operational_capacity_sandwich`	Positive-slack sufficiency and zero-slack conservation-law necessity under a load-certified model.

The Lean theorem names are identifiers, not paper theorem numbers. The artifact package includes a consolidated upload file and build log so the names can be searched directly.

EC.5. Statewise Feasible Families and Operational Boundaries

The statewise corollary is a conditioning argument rather than a new drift identity. Let $\chi(t)$ record scheduler-visible information such as alive nodes, pinned tasks, queued jobs, and measured admission status. If the candidate family $\mathcal{A}^{\mathrm{cand}}(Q(t),\chi(t))$, lower-service table $\mu^{\mathrm{L}}(Q(t),\chi(t),\cdot)$, penalty bounds, oracle bounds, and moment bounds satisfy the same constants for every admitted $(Q(t),\chi(t))$, then the proof in EC.1–EC.4 holds after conditioning on $(Q(t),\chi(t))$. Taking expectation over $\chi(t)$ preserves the drift bound because the constants are uniform over the admitted family.

The operational capacity statement is kept separate. Sufficiency follows from the positive-slack drift chain above once a stochastic model certifies the arrival mean, service accounting, lower-service domination, moment bounds, and local recurrence conditions. Necessity is not obtained by defining a model to "stabilize" a load; it requires the model to encode the load and obey a conservation law showing that long-run departures cannot exceed the capacity base. This separation is essential for the paper's claim boundary: Lean audits the implication from certified assumptions to drift and recurrence, while the experiments and admission gates certify when Scheduleurm rows satisfy those assumptions.

EC.6. Lean Crosswalk and Artifact Assumptions

Table 13 maps the paper-readable statements to the Lean identifiers that audit them. Table 14 separates formal hypotheses from empirical certificates.

Table 14 Mathematical assumptions and Scheduleurm certificates.

Object	Mathematical role	Artifact or empirical certificate
λ, δ	Arrival mean and full-action capacity slack.	Online arrival replay, load sweep, and finite lower-service capacity gates.
ϵ_{cand} or $L\rho$	Candidate-set or fabric-cover support loss.	Measured finite-slice support gaps, declared bucket admission, and fabric-cover calibration when available.
μ_t^{L}, ϵ_{est}	Conservative lower-service rows and estimation loss.	Holdout lower confidence bound gates, measured service cache, and admission/probe status.
P_0, β	Bounded and queue-scaled penalty consumption.	Penalty-unit ledger, guard/tie-break audits, and bounded-switching configuration.
α_0, α_1	Additive and queue-scaled approximate oracle loss.	Oracle audit comparing selected action against theorem-facing candidate families.
B, N, γ	Second-order moment bound and finite-set drift threshold.	Workload bounds, controlled trace statistics, and recurrence-side finite-state admission conditions.
Statewise $\chi(t)$	Scheduler-visible feasible-family index.	Alive-node, pinned-job, queued-job, service-cache, and admission-state trace fields.
Operational model certificate	Links stochastic arrivals and services to the queueing theorem.	Load-certified model contract plus conservation-law boundary; not inferred from Lean alone.

The table is deliberately conservative: Lean verifies the implications once these hypotheses hold, while the Scheduleurm artifact determines which measured rows satisfy them.

Appendix: Gate Ladder

Tables 15 and 16 are the compact printed version of `md/gate_status_dashboard.md`. Each row distinguishes the scoped claim from the adjacent strong claim and lists the threshold required to promote the stronger statement.

Table 15 Reviewer gate ladder for scoped and adjacent strong claims.

Gate	Scoped ready	Strong ready	Blocker	Next threshold
Stability theorem	true	false	Operational claims require a matched stochastic/load certificate.	Keep theorem names, build log, and no-sorry audit synchronized with the final manuscript.
Declared finite-domain cover	true	false	Unmeasured states are probe/defer before positive theorem admission.	Add service-cache v2 timing metadata and perturbation profiling for broader universes.
All-state safety cover	true	false	Unknown states have zero theorem service until measured.	Measure and admit future buckets or define a finite all-state universe.
Strict production admission	true	false	Future unmeasured jobs must remain outside the theorem stream.	Keep strict admission in trace-only production telemetry.
Gavel adapter certificate	true	false	Adapter/native simulator readiness is not scalar service-unit equivalence.	Keep scalar and profile-aware certificates separate.
Gavel profile-aware calibration	true	false	Profile-aware same-workload model passes on bounded q01/q11 windows, but scalar calibration has p95 relative error about 0.5.	Do not claim scalar service-unit equivalence or full-stack superiority.
Gavel resident-delay/JCT holdout	true	false	Immediate co-location beats defer on the controlled resident rows even under a resident-alone challenge rate.	Treat as measured-service holdout, not direct full-stack Gavel execution.
Gavel native same-workload comparison	true	false	q01 ResNet-50 full-taskset native simulator row favors Scheduleurm by 7.30× makespan and 6.87× mean-JCT/flow in raw simulator units.	Treat as scoped native-simulator evidence; keep it separate from scalar service-unit equivalence.
Gavel physical same-workload gate	true	false	The same temporary CUDA probe completes through Gavel's scheduler/worker/remote procedure call (RPC)/`GavelIterator` path, and the direct native path has lower JCT on the same host.	Treat as a scoped Gavel physical-runtime row, not as every Gavel workload or multi-node Gavel superiority.
Named external runtime gate	true	true	Gavel, Pollux/AdaptDL, Sia, IADeep, and Salus all have named same-host same-workload full-stack rows with paired native superiority.	Keep the claim limited to the named systems, measured probes, same-host substrate, and artifacted compatibility fixes; do not generalize to arbitrary systems, future workloads, or production claims outside the strict history population.
Registered external adapter universe	true	false	All 11 non-adjacent registered external scheduler families have policy/service-unit adapter rows in the measured-cache Scheduleurm+external union, and the measured-cache external-policy frontier is closed; 5/11 also have direct same-workload binary rows.	This is finite adapter closure, not external-binary full-stack superiority for every registered or arbitrary SOTA.
External-policy action-union gate	true	false	External policy-family actions are included as measured-cache configuration-trajectory actions; metric-specific union rows close external-policy envelopes within tolerance.	Do not read policy-union dominance as strict all-quadrant dominance or direct binary/full-stack superiority.
External-policy algorithm-upgrade gate	true	false	Adaptive scalarized union, state-dependent marginal service, global lookahead, ETA reuse, and expanded external-policy action families are certified as opt-in replay artifacts.	Do not treat as production-default launch evidence or direct external full-stack superiority.
External native execution attempts	true	true	Gavel physical, Pollux/AdaptDL, Sia AdaptDL/Kubernetes, IADeep extender, and Salus server/zrpc rows are scoped same-host full-stack evidence; Decima is simulator-only.	Keep the strong claim limited to the named measured probes; do not generalize to arbitrary systems, future workloads, or production-wide traces.
Decima same-domain benchmark	true	false	Paired Spark-DAG simulator runs complete for fixed seeds; dynamic-partition wall time is noninferior but mean-completion performance is mixed.	Treat as adjacent-domain execution evidence, not GPU co-location full-stack superiority.

The machine-readable dashboard is included in the artifact package.

Table 16 Reviewer gate ladder, continued.

Gate	Scoped ready	Strong ready	Blocker	Next threshold
Online/ablation summary	true	false	Replay-policy semantics are not live external binary runs.	Keep replay-policy semantics separate from live external execution and stochastic lower-service certification.
Corner-case lower-service gate	true	false	Five stable node007/node001 corner-case rows enter a standalone service cache with positive row-level lower-service slack.	Do not merge into production-wide or all-state claims without explicit admission.
Selected-profile stochastic LCB	true	false	Aggregate-window sample thresholds are met and the LCB lower-service capacity gate has $\delta_{\mathrm{LCB}} = 0.024712$; absolute and diagonal mean-service eta remain negative.	Claim only stochastic lower-service capacity, not 0.8 mean-service certification.
Controlled launched completion	true	true	32 controlled theorem-admitted tasks launch and complete with $\alpha_0 = \alpha_1 = 0$; strict scheduler-history organic completion is also closed.	Keep controlled and organic evidence paths reported separately.
Organic canary recorder	true	true	Live-trace completion counters remain false, but strict history completion passes with 4,020 launched rows, 3,992 completed rows, 35 domains, 13 nodes, and no strict unadmitted launches.	Do not read this as raw-history or future-workload closure.
Production launch/completion	true	true	The rolling safe-launch gate now has active progress plus shadow trace and the strict scheduler-history launched-completion certificate.	Keep live oracle trace closure separate from history completion.
Multi-node theorem shadow	true	false	Cross-node theorem-candidate shadow sees two reachable GPU nodes, 18 certified candidate rows, 626 configurations, and a selected two-node robust-MaxWeight configuration.	This is read-only candidate-family evidence, not launched original multi-node full-stack superiority.
Production organic readiness bridge	true	true	Strict admission, canary recorder, active-production theorem shadow, queued theorem trace, and history-completion snapshot are closed.	Do not claim every live slot has an emitted oracle trace row.
Global dispatcher prototype	true	false	Prototype is wired only as an opt-in scheduler soft-hint A/B hook; it is not the live default and does not by itself certify launched global-action traces.	Run controlled launches with `global_theorem_maxweight_v1` and collect launched global-action theorem traces.
Optional algorithm upgrade	true	false	Global batch candidate construction, state-dependent marginal service lookup, online LCB/ETA, backlog-aware guarded replay, and q01 co-location validation pass with 0 regressions and 4 improving task sets under 0.5% replay tolerance.	Collect controlled launched global-action traces before using production-default language.
Non-future closure gate	true	true	Excluding arbitrary future workloads, the finite named external-system, registered policy-semantics, production history, Scheduleurm-native multi-node history, and Decima bridge rows all close.	Keep arbitrary future workloads and stronger unbounded external-binary extensions explicitly excluded.
Reviewer environment manifest	true	false	No clean Docker/Nix container is provided.	Run the full reproduction script inside a clean container or Nix environment.

The machine-readable dashboard is included in the artifact package.

Code and Data Availability

The code, experiment artifacts, measured service-cache summaries, reviewer supplement manifest, and README files for this submission are provided in the supplementary Scheduleurm artifact package. The package contains the Scheduleurm source tree, the `algorithm/` and `simulation/` replay modules, the generated gate reports under `md/`, and the Lean supplement headed by `ScheduleurmUpload.lean`. The script `scripts/reproduce_or_submission.sh` regenerates the gate dashboard, safe launch/completion gates, Gavel calibration gate, organic canary recorder, reviewer environment manifest, online/ablation summary, targeted tests, Portable Document Format (PDF) build, and `reproduction_manifest_20260612.json`; it does not pass `-allow-launch`. Raw scheduler history records that are not part of the theorem-facing completed-active population are retained as operational telemetry and are not used as the claimed replication population.

References

Altman E, Avrachenkov K, Ayesta U (2006) A survey on discriminatory processor sharing. *Queueing Systems* 53(1–2):53–63, URL http://dx.doi.org/10.1007/s11134-006-7586-8.

Bekker R, Borst SC (2006) Optimal admission control in queues with workload-dependent service rates. *Probability in the Engineering and Informational Sciences* 20(4):543–570, URL http://dx.doi.org/10.1017/S0269964806060335.

Carvalho MNL, Simitsis A, Queralt A, Romero O (2024) Workload placement on heterogeneous CPU-GPU systems. *Proceedings of the VLDB Endowment* 17(12):4241–4244, URL http://dx.doi.org/10.14778/3685800.3685845.

Chen W, Mo Z, Xu H, Ye K, Xu C (2023) Interference-aware multiplexing for deep learning in GPU clusters: A middleware approach. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (ACM), URL http://dx.doi.org/10.1145/3581784.3607060.

Dai JG, Gluzman M (2022) Queueing network controls via deep reinforcement learning. *Stochastic Systems* 12(1):30–67, URL http://dx.doi.org/10.1287/stsy.2021.0081.

Dai JG, Meyn SP (1995) Stability and convergence of moments for multiclass queueing networks via fluid limit models. *IEEE Transactions on Automatic Control* 40(11):1889–1904, URL http://dx.doi.org/10.1109/9.471210.

de Moura L, Ullrich S (2021) The Lean 4 theorem prover and programming language. URL https://arxiv.org/abs/2107.00608.

Gupta V, Zhang J (2022) Approximations and optimal control for state-dependent limited processor sharing queues. *Stochastic Systems* 12(2):205–225, URL http://dx.doi.org/10.1287/stsy.2021.0087.

Le TN, Sun X, Chowdhury M, Liu Z (2020) AlloX: Compute allocation in hybrid clusters. *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16 (ACM), URL http://dx.doi.org/10.1145/3342195.3387547.

Liu B, Xie Q, Modiano E (2022) RL-QN: A reinforcement learning framework for optimal control of queueing systems. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 7(1):1–35, URL http://dx.doi.org/10.1145/3529375.

Mahajan K, Balasubramanian A, Singhvi A, Venkataraman S, Akella A, Phanishayee A, Chawla S (2020) Themis: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 289–304 (USENIX Association), URL https://www.usenix.org/conference/nsdi20/presentation/mahajan.

Narayanan D, Santhanam K, Kazhamiaka F, Phanishayee A, Zaharia M (2020) Heterogeneity-aware cluster scheduling policies for deep learning workloads. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*, 481–498 (USENIX Association), URL https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak.

Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP (2021) Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 1–18 (USENIX Association), URL https://www.usenix.org/conference/osdi21/presentation/qiao.

Scully Z, Harchol-Balter M, Scheller-Wolf A (2020) Simple near-optimal scheduling for the M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4(1):1–29, URL http://dx.doi.org/10.1145/3379477.

Stolyar AL (2004) MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *The Annals of Applied Probability* 14(1):1–53, URL http://dx.doi.org/10.1214/aoap/1075828046.

Subramanya SJ, Arfeen D, Lin S, Qiao A, Jia Z, Ganger GR (2023) Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 642–657 (ACM), URL http://dx.doi.org/10.1145/3600006.3613175.

Tassiulas L, Ephremides A (1992) Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, URL http://dx.doi.org/10.1109/9.182479.

Xiao W, Bhardwaj R, Ramjee R, Sivathanu M, Kwatra N, Han Z, Patel P, Peng X, Zhao H, Zhang Q, Yang F, Zhou L (2018) Gandiva: Introspective cluster scheduling for deep learning. *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, 595–610 (USENIX Association), URL https://www.usenix.org/conference/osdi18/presentation/xiao.

Yang Z, Srikant R, Ying L (2023) Learning while scheduling in multi-server systems with unknown statistics: MaxWeight with discounted UCB. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4275–4312 (PMLR), URL https://proceedings.mlr.press/v206/yang23d.html.

Yu P, Chowdhury M (2020) Salus: Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, volume 2, 600–615, URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/d9cd83bc91b8c36a0c7c0fcca59228f2-Abstract.html.